

TD Chapitre 2 : Organisation Mémoire

Pointeurs, Tableaux et Forensics Mémoire

Dr. El Hadji Bassirou TOURÉ

Département de Mathématiques et Informatique

Université Cheikh Anta Diop de Dakar

Exercice 1 : Endianness et Représentation Mémoire

Partie A : La valeur 0x12345678 est stockée à l'adresse 0x100 sur 4 octets.

- a) Dessiner le contenu de la mémoire (adresses 0x100 à 0x103) en **big-endian**.
- b) Dessiner le contenu de la mémoire en **little-endian**.
- c) Sur une machine little-endian, si on lit l'octet à l'adresse 0x100, quelle valeur obtient-on ?
- d) Sur une machine little-endian, si on lit un **short** à l'adresse 0x100, quelle valeur obtient-on ?

Partie B : Voici un dump mémoire d'une machine little-endian :

Addr	+0	+1	+2	+3
0x200	78	56	34	12

- e) Quelle est la valeur de l'entier 32 bits stocké à 0x200 ?
- f) Si ce même dump provenait d'une machine big-endian, quelle serait la valeur ?

Exercice 2 : Pointeurs et Arithmétique

Soit le code suivant sur une machine 32 bits :

```
int arr[5] = {10, 20, 30, 40, 50};
/* &arr[0] = 0x1000 */

int *p = arr;
int *q = &arr[3];
```

- a) Quelle est la valeur de p ?
- b) Quelle est la valeur de q ?
- c) Quelle est la valeur de p + 2 ?
- d) Quelle est la valeur de q - p ? (attention au type !)
- e) Quelle est la valeur de *(p + 4) ?
- f) Quelle est la valeur de p[2] ? Équivalent à quelle expression ?
- g) Quelle est la valeur de (char*)q - (char*)p ? (résultat en octets)

Partie B : Soit void *base = (void *)0x1000;

Calculer les adresses résultantes :

Expression	sizeof(*p)	Adresse résultante
(int *)base + 3	4	
(short *)base + 3	2	
(char *)base + 3	1	
(double *)base + 2	8	

Exercice 3 : Chaînes de Caractères

Partie A : Soit le dump mémoire suivant :

Addr	+0	+1	+2	+3	+4	+5	+6	+7
0x20	42	6F	6E	6A	6F	75	72	00
0x28	55	43	41	44	00	00	00	00

- Convertir les octets 0x20 à 0x27 en caractères ASCII.
- Si `char *s1 = (char *)0x20`, quelle est la chaîne pointée ?
- Quelle est la valeur de `strlen(s1)` ?
- Quelle est la valeur de `sizeof("Bonjour")` ?
- Expliquer la différence entre `strlen` et `sizeof` pour les chaînes.

Partie B : Comparer les deux déclarations :

```
char arr[] = "Hello";  
char *ptr = "Hello";
```

- Quelle est la valeur de `sizeof(arr)` ?
- Quelle est la valeur de `sizeof(ptr)` sur une machine 64 bits ?
- L'instruction `arr[0] = 'X';` est-elle valide ?
- L'instruction `ptr[0] = 'X';` est-elle valide ? Pourquoi ?
- L'instruction `ptr = "World";` est-elle valide ? Et `arr = "World";` ?

Table ASCII : 0x00=NUL, 0x41='A', 0x42='B', 0x43='C', 0x44='D', 0x55='U', 0x6A='j', 0x6E='n', 0x6F='o', 0x72='r', 0x75='u'

Exercice 4 : Allocation Dynamique

Partie A :

```
int *a = malloc(5 * sizeof(int));  
int *b = calloc(5, sizeof(int));
```

- Combien d'octets sont alloués pour `a` ? Pour `b` ?
- Quelle est la différence entre les contenus initiaux de `a` et `b` ?
- Pourquoi est-il dangereux de lire `a[0]` juste après le `malloc` ?
- Écrire le code pour libérer correctement la mémoire.

Partie B : Pour chaque code, identifier l'erreur et expliquer la conséquence :

```
/* Cas 1 : Use-after-free */  
int *p = malloc(sizeof(int));  
free(p);  
*p = 42;  
  
/* Cas 2 : Memory leak */  
int *q = malloc(10 * sizeof(int));  
q = malloc(20 * sizeof(int));  
  
/* Cas 3 : Double free */  
int *r = malloc(sizeof(int));  
free(r);  
free(r);
```

Exercice 5 : Forensics Mémoire I (32 bits)

Soit le dump mémoire suivant :

Addr	+0	+1	+2	+3	+4	+5	+6	+7
0x00	1e	00	00	00	00	00	00	00
0x08	aa	bb	cc	dd	ee	ff	00	11
0x10	8a	7c	6f	22	9a	66	44	17
0x18	33	77	6f	6c	66	62	79	74
0x20	65	73	00	44	00	00	88	c0
0x28	de	ad	be	ef	ca	fe	f0	0d

Sachant que :

```
char *c = (char *)0x19;  
short *s = (short *)0x0C;  
int *p = (int *)0x28;
```

Remplir le tableau avec le **type C** et la **valeur hexadécimale** :

Expression C	Type C	Valeur Hex
*s		
*p		
*(c+2)		
p[1]		
*((short *) (c-1))		
strlen(c)		

Exercice 6 : Forensics Mémoire II — Double Indirection

Machine 32 bits, little-endian. Dump mémoire :

Addr	+0	+1	+2	+3	+4	+5	+6	+7
0x1000	0A	00	00	00	14	00	00	00
0x1008	1E	00	00	00	00	10	00	00
0x1010	0C	10	00	00	00	00	00	00

Variables :

```
int arr[3];          /* &arr[0] = 0x1000 */  
int *p;              /* &p = 0x100C */  
int **pp;            /* &pp = 0x1010 */
```

- Donner les valeurs de `arr[0]`, `arr[1]`, `arr[2]`.
- Quelle est la valeur de `p` ? Vers quelle adresse pointe-t-il ?
- Quelle est la valeur de `*p` ?
- Quelle est la valeur de `pp` ? Vers quelle adresse pointe-t-il ?
- Quelle est la valeur de `*pp` ?
- Quelle est la valeur de `**pp` ?
- Dessiner un schéma montrant les pointeurs et leurs cibles.

Rappels Essentiels

Endianness :

- **Little-endian** (x86) : LSB à l'adresse basse
- **Big-endian** : MSB à l'adresse basse

Arithmétique des pointeurs :

- $p + n$ ajoute $n \times \text{sizeof}(*p)$ octets
- $p[i] \equiv *(p + i)$
- $q - p$ = nombre d'éléments entre p et q

Chaînes :

- Terminées par `'\0'` (0x00)
- `strlen` ne compte pas le terminateur
- `sizeof` compte le terminateur

Valeurs suspectes en debug :

- 0xDEADBEEF, 0xCAFEFABE : marqueurs de debug
- 0xCDCDCDCD : mémoire non initialisée (MSVC)
- 0xDDDDDDDD : mémoire libérée (MSVC)